



DBpedia Databus Python Client: Manifest-Driven Reproducible Workflow Orchestration

GSoC 2026 Project Proposal

Organization: DBpedia

Contact information:

Full name: Dhanashree Santosh Petare

Degree: Bachelor of Technology in Computer Engineering (B. Tech)

University: Vishwakarma Institute of Technology (VIT), Pune

Location: Pune, Maharashtra, India

Time Zone: Indian Standard Time (UTC+5.30)

Contact: Email : ghanashreepetare@gmail.com

GitHub : [DhanashreePetare](https://github.com/DhanashreePetare)

LinkedIn : [DhanashreePetare](https://www.linkedin.com/in/DhanashreePetare)

Resume: [Link](#)

Introduction

Hi, I'm Dhanashree Petare, and I'm currently in my second year of BTech in Computer Science at Vishwakarma Institute of Technology, Pune, with a CGPA of 9.2/10.

I have hands-on experience building data pipelines, CLI tooling, and backend systems. Here are a couple of my recent projects:

1. [Skill-O-Hire – AI-Powered Employment Bridge](#): Built an AI-driven platform connecting job seekers, employers, and NGOs, implementing GenAI-based resume **parsing**, structured skill **extraction**, and personalized **recommendation pipelines**. Designed and integrated REST APIs for resume processing, candidate-employer matching workflows, and scalable backend services for automated evaluation and data handling.
2. [AgentPlay Chrome Extension](#): Built an AI-powered **Chrome extension** that converts YouTube videos into structured learning modules with automated summarization, Q&A generation, and multilingual audio synchronization using a Python/Flask backend with Mistral AI and Google Gemini API integration.

I have a strong understanding of Python, C/C++, JavaScript, and Scala, and am well-versed in Semantic Web technologies including RDF, SPARQL, JSON-LD, and Linked Data principles, with practical experience in CLI tooling, REST APIs, metadata validation, authentication workflows, and data format handling including RDF dumps and compression pipelines — all directly relevant to this project.

Past Open Source Experience

My open source journey began through **Hacktoberfest 2025**, where I earned the Super Contributor badge with **6+ pull requests merged** across different community repositories. Contributions included implementing convolutional layers for image processing in **PyDeepFlow**, replacing unsafe `exec()` calls with subprocess-based execution and adding a resizable output console with live run logs in **NodeBox** — a frontend-facing improvement requiring understanding of the existing UI architecture — and extending LLM model support in **AI-Robo-Advisor**. That experience taught me how to navigate unfamiliar codebases, work within contributor guidelines, and communicate effectively through pull requests and reviews.

Following that, I began contributing actively to the **DBpedia ecosystem** — specifically to the Databus Python Client, the DBpedia Extraction Framework, and the DBpedia Chatbot. These contributions gave me deep familiarity with the client's internal architecture, authentication workflows, and compression pipelines, which directly motivated this proposal. The detail of this work is covered in the Pre-GSoC Contributions section below.

Time Commitment

I can dedicate 6 to 7 hours per day to the project, amounting to 40–45 hours per week. I am open to adjusting this based on project needs and mentor availability. I have no academic commitments during the summer period that would conflict with GSoC, and I am flexible with working hours to align with my mentor's time zone at the DBpedia Foundation. I am comfortable with any means of communication and will ensure consistent updates and availability throughout the project.

Pre-GSoC Evaluation Tasks

I reached out to the mentor early and have been actively contributing to the Databus Python Client and the broader DBpedia ecosystem since December 2025, which directly informed the design and scope of this proposal.

Merged pull requests:

1. **Release databusclient v0.15 to PyPI** — Full production release including version bump, changelog, and test verification [#41](#)
2. **On-the-fly Compression Conversion during Download** — CLI-based bz2/gz/xz format conversion with automatic detection and recompression pipeline [#45](#)
3. **Vault Authentication & Error Handling Improvements** — Host-restricted Vault token exchange, early failure detection, and auth-flow test coverage [#40](#)
4. **Extraction Framework Robustness** — Dynamic namespace prefix detection to prevent extraction crashes; improved parser safety [#812](#)
5. **DBpedia Chatbot Enhancements** — Improved conversation sorting and response handling logic for better stability [#62](#)

Project Description

The **Databus Python Client** is a command-line and Python library tool for managing datasets on the DBpedia Databus — a centralized platform for publishing and retrieving structured RDF data and Knowledge Graphs. It supports three core operations: **downloading** datasets with compression handling and authentication, **deploying** datasets with rich metadata, and **deleting** datasets or specific versions programmatically.

Today, chaining these operations into a reproducible pipeline requires manual command execution, with no structured record of what was run and no way to reproduce a past operation without reconstructing every parameter from scratch.

This project introduces three enhancements — **a format and mapping conversion layer, a structured run manifest system, and a declarative workflow engine** — all fully backward compatible and built entirely on top of existing client logic.

Project Milestones

1. **Format and Mapping Conversion Layer** — implement Layer 2 (format conversion) and Layer 3 (mapping conversion) using rdflib, bringing the Python client to feature parity with the Java client. Includes `--convert-format`, `--graph-name`, and `--base-uri` CLI flags with 14 round-trip tests.
2. **Structured Run Manifest System** — introduces a fixed-format JSON-LD schema, a `ManifestContext` object threaded through all three existing commands, and `--manifest` flag support across download, deploy, and delete.
3. **Manifest Replay and Summary** — enable any previous operation to be re-executed exactly from its saved manifest, with structured error and traceback capture in the manifest and a summary command that builds on this information.
4. **Declarative Workflow Engine** — a YAML-based pipeline system orchestrating download, deploy, and delete steps sequentially with dynamic step chaining via `${steps.name.output_files}` resolution and per-step error recovery supporting fail, continue, and retry modes.
5. **Workflow-Manifest Integration and Example Workflows** — five fully tested real-world workflows covering reproducible research, automated publishing, batch deployment, CI/CD integration, and failure debugging.

Understanding the Existing Codebase

The Databus Python Client is cleanly structured around three independent API modules — `download.py`, `deploy.py`, and `delete.py` — each exposing a single core function that is stateless, exception-based, and mutually independent. This architecture makes the manifest system and workflow engine implementable without rewriting any existing logic.

The existing module structure is as follows:

```
databusclient/
├── cli.py                ← Click CLI entry point
                        three command groups:
                        download, deploy, delete
├── api/
│   ├── download.py       ← download() - 11 parameters
│   ├── deploy.py         ← deploy(dataid, api_key)
│   │                     create_dataset() builds dataid
│   ├── delete.py         ← delete(databusURIs, databus_key,
│   │                     dry_run, force)
│   ├── utils.py          ← SHA256, HTTP, URI parsing
├── extensions/
│   └── webdav.py          ← optional WebDAV integration
```

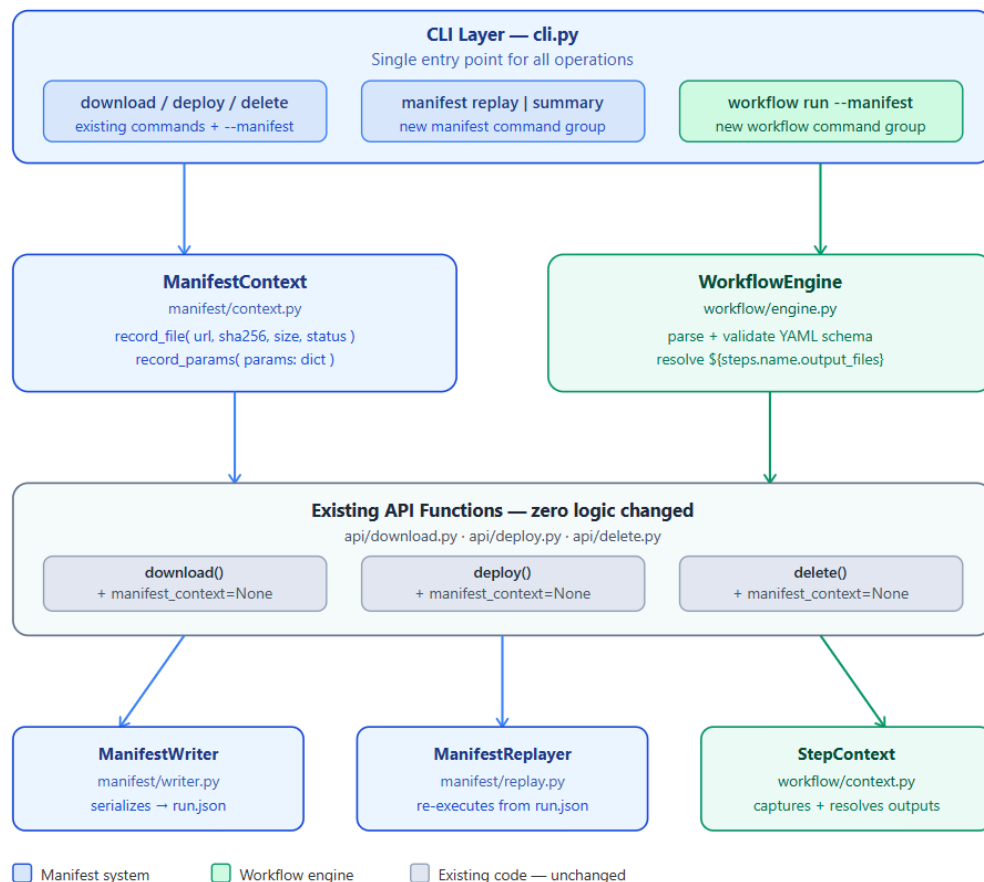
Three properties of this codebase directly shape the implementation approach:

- **Complete statelessness.** All three functions operate independently with no shared state. Threading a manifest context through them requires adding exactly one optional parameter to each — zero refactoring of existing logic.
- **Exception-based error handling.** `download()` raises `DownloadAuthError`, `deploy()` raises `DeployError` and `BadArgumentException`, and `delete()` raises `requests.RequestException`. The workflow engine's per-step error recovery wraps each step in try/except and handles these per the configured `on_error` mode.
- **Two-stage deploy construction.** `deploy()` takes a pre-built `dataid` dict constructed by `create_dataset()` from user inputs. For manifest replay, the pre-construction inputs — `version_id`, `title`, `abstract`, `description`, `license_url`, `distributions` — must be saved rather than the computed `dataid`. On replay, `create_dataset()` is called again to reconstruct it before calling `deploy()`.

Overall Architecture

The diagram below shows how both systems sit on top of the existing client without disturbing it.

Fig. 1 — System Architecture: Manifest System and Workflow Engine Integration



The CLI layer remains the single entry point for all operations. Existing commands gain an optional `--manifest` flag. Two new command groups are added: `manifest` for replay and summary operations, and `workflow` for running YAML pipeline files.

When `--manifest` is provided, a `ManifestContext` object is created and passed into the existing API function as an optional parameter. The API function records file-level outcomes into the context as it runs — existing logic is completely untouched. At completion, `ManifestWriter` serializes the context to a fixed-format JSON-LD file.

The workflow engine follows the same path but adds orchestration on top. It parses the YAML pipeline file, creates a shared `StepContext` for passing outputs between steps, and executes each step by calling the same existing API functions. A `ManifestContext` runs alongside the entire workflow, recording every step's outcome into a single unified manifest.

The key architectural principle throughout is that **existing API functions are never rewritten — they are only wrapped and observed. Every new capability is additive.**

1. Format and Mapping Conversion Layer (Milestone 1)

1.1 Overview

The existing **Java Databus Client** [2] implements a four-layer data pipeline — Download, Compression, Format, and Mapping — as described in Frey et al. [1]. The Python client currently implements Layer 0 (Download) and Layer 1 (Compression). This project extends it with Layer 2 (Format conversion) and Layer 3 (Mapping conversion), bringing the Python client to feature parity with the Java client for the first time.

With these two layers in place, a user can download any Databus dataset and receive it in exactly the format their application needs — regardless of how it was originally published — in a single command.

1.2 The Four-Layer Pipeline

Layer 0 — Download and Persistence	← already implemented
Layer 1 — Compression	← already implemented
Layer 2 — Format conversion	← this project
Layer 3 — Mapping conversion	← this project

After download and decompression, the Format Layer converts between serialization formats within the same equivalence class. If the input and output formats belong to different equivalence classes, the Mapping Layer performs the cross-class transformation. The result is then recompressed and written to disk.

1.3 Equivalence Classes

Three equivalence classes are defined — groups of formats that represent the same amount of information and can be losslessly converted between each other:

Equivalence Class	Supported Formats
RDF Quads	nquads, trig, trix, json-ld
RDF Triples	ntriples, turtle, rdf-xml
Tabular	csv, tsv

Conversions within an equivalence class are handled by the Format Layer. Conversions between equivalence classes are handled by the Mapping Layer.

1.4 Layer 2 — Format Conversion

The Format Layer converts between serialization formats within the same equivalence class — for example N-Triples to Turtle, or N-Quads to JSON-LD. Conversion within a class is lossless since all formats in the class share the same internal data model.

rdflib — already a dependency of the Python client — natively supports all three equivalence classes and all listed formats. No new dependencies are required.

```
databusclient download <URI> --convert-format turtle
databusclient download <URI> --convert-format ntriples
databusclient download <URI> --convert-format nquads
databusclient download <URI> --convert-format csv
```

1.5 Layer 3 — Mapping Conversion

The Mapping Layer handles conversions between different equivalence classes. Four mapping directions are supported:

➤ RDF Triples → RDF Quads

Assigns a configurable graph name to all triples, promoting them to named graph quads via **--graph-name**.

```
databusclient download <URI> \
  --convert-format nquads \
  --graph-name https://myorg.org/graph/labels
```

➤ RDF Quads → RDF Triples

Splits a quads file into multiple triple files — one per named graph — each named after its graph identifier.

➤ RDF → Tabular (CSV/TSV)

Maps each RDF subject to a row in a wide table. The subject IRI goes in the first column and each property becomes a column header. A reverse mapping file is generated alongside the output to enable round trip validation.

➤ Tabular → RDF

Generic strategy: column headers become predicate URIs by appending them to a configurable base URI. Each row becomes a subject resource. Cell values are typed as IRIs if parseable as valid URIs, otherwise as string literals.

```
databusclient download <URI> \  
  --convert-format ntriples \  
  --base-uri https://myorg.org/data/
```

1.6 Round Trip Tests

Following the evaluation strategy from Frey et al. [1], two types of round trip tests validate correctness:

1. **Format round trip:** Read a file, serialize to the same format, parse again, and compare graphs for isomorphism. Validates parser and serializer consistency within an equivalence class.
2. **Mapping round trip:** Convert $A \rightarrow B \rightarrow A$, then compare the original and reconstructed graphs for information equality. For non-bijective mappings such as $\text{RDF} \rightarrow \text{CSV}$ where some RDF expressiveness is lost, the comparison accounts for predictable information loss — referred to as "quasi-equal."

Required test coverage: **9 format round trip tests** and **5 mapping round trip tests**, following the formula defined in the reference implementation.

1.7 Relation to Existing Compression Conversion

The existing `--convert-to` and `--convert-from` flags handle Layer 1 (compression). The new `--convert-format` flag handles Layers 2 and 3. They are independent and composable:

```
# Download, convert format, and recompress  
databusclient download <URI> \  
  --convert-format turtle \  
  --convert-to gz
```

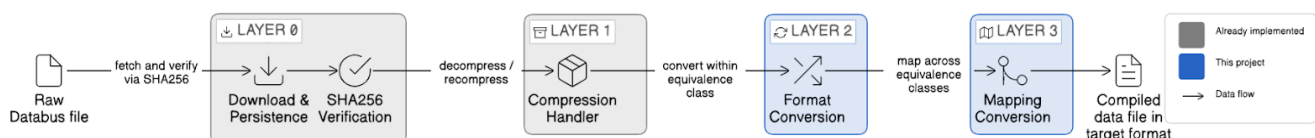


Fig. 2 — Four-Layer Data Compiling Pipeline

2. Manifest System (Milestone 2, 3)

The manifest system introduces an optional `--manifest` flag to all three existing commands. When provided, every detail of the operation — input parameters, resolved file URLs, checksums, compression settings, and per-file status — is recorded into a fixed-format JSON-LD file. When not provided, behavior is completely unchanged. Two active capabilities are built on top of the recorded data:

1. `manifest replay` re-executes any previous operation exactly by reading the saved `replay_params`. For `deploy`, pre-construction inputs are saved so `create_dataset()` is called again on replay rather than restoring the computed `dataid`. For `delete`, replay always defaults to `dry_run=True` as a safety guard.
2. `manifest summary` reads the existing summary block from the manifest and prints it as readable console output — no new data collected, no new file generated.

2.1 Manifest JSON-LD Schema

The manifest is serialized as JSON-LD using the DataID vocabulary — the same vocabulary used by the Databus platform itself. This makes manifests semantically interoperable with the broader DBpedia ecosystem and queryable via SPARQL. The schema is fixed and versioned from day one via `dbus:schemaVersion` so future additions never break existing files. No sensitive credentials are ever written.

```
{
  "@context": {
    "dataid": "http://dataid.dbpedia.org/ns#",
    "dcat": "http://www.w3.org/ns/dcat#",
    "dcterms": "http://purl.org/dc/terms/",
    "xsd": "http://www.w3.org/2001/XMLSchema#",
    "dbus": "http://databus.dbpedia.org/manifest/ns#"
  },
  "@type": "dbus:OperationManifest",
  "dbus:schemaVersion": "1.0",
  "dbus:clientVersion": "0.15",
  "dbus:command": "download",
  "dbus:authMethod": "vault_token",
  "dcterms:issued": {
    "@value": "2024-03-24T10:00:00Z",
    "@type": "xsd:dateTime"
  },
  "dbus:endpoint": "https://databus.dbpedia.org",
  "dbus:replayParams": {
    "databusURIs": [
      "https://databus.dbpedia.org/dbpedia/generic/labels/2023.12.01"
    ],
    "localDir": "./data",
    "convert_to": "bz2",
    "all_versions": false,
    "validate_checksum": true
  },
  "dataid:distribution": {
    "@type": "dataid:Distribution",
    "dataid:file": [
      {
```

```

    "@type": "dataid:File",
    "dcat:downloadURL": "https://downloads.dbpedia.org/2023.12.01/labels_en.ttl.bz2",
    "dataid:checksum": "abc123...",
    "dataid:byteSize": 104857600,
    "dataid:compression": "bz2",
    "dbus:status": "success",
    "dbus:downloadedAt": {
      "@value": "2024-03-24T10:02:33Z",
      "@type": "xsd:dateTime"
    }
  }
]
},
"dbus:executionResult": {
  "@type": "dbus:ExecutionSummary",
  "dbus:totalFiles": 1,
  "dbus:succeeded": 1,
  "dbus:failed": 0,
  "dbus:totalBytes": 104857600
}
}

```

The `dbus:replayParams` field is the key design decision here. It stores the exact CLI-level inputs needed to reconstruct any operation — not the computed outputs. This distinction matters: saving the computed `dataid` for deploy would break replay if file paths or metadata change between runs. Saving the pre-construction inputs means replay always reconstructs correctly from source.

When an operation fails, the manifest records structured error details directly in the affected `dataid:File` entry — including the error message, traceback, and retry count. This is what makes the manifest genuinely useful for debugging rather than just logging.

```

{
  "@type": "dataid:File",
  "dcat:downloadURL": "https://downloads.dbpedia.org/.../labels_en.ttl.bz2",
  "dbus:status": "failed",
  "dbus:errorMessage": "ConnectionTimeout: failed after 3 retries",
  "dbus:errorTraceback": "Traceback (most recent call last):\n  ...",
  "dbus:retryCount": 3
}

```

2.2 How Context Threads Through Existing Functions

The change to each existing API function is a single optional parameter. All existing logic is completely untouched:

```

def download(
    localDir: str,
    databusURIs: List[str],
    convert_to=None,
    ...
    manifest_context=None,      # ← only addition
) -> None:
    for databusURI in databusURIs:
        # all existing logic unchanged
        file_url, sha256, size = _download_file(databusURI)

        if manifest_context:

```

```
manifest_context.record_file(
    url=file_url,
    sha256=sha256,
    size_bytes=size,
    status="success"
)
```

The same pattern applies identically to `deploy()` and `delete()`. When `--manifest` is not passed, `manifest_context` is `None` and the condition never fires — zero performance impact on existing usage.

2.3 Manifest Summary Output

```
Command      : download
Executed     : 2024-03-24 10:00:00 UTC
Endpoint     : https://databus.dbpedia.org
Auth         : vault_token
Files        : 1 succeeded · 0 failed
Total        : 100.0 MB
Status       : completed
```

3. Workflow Engine (Milestone 4, 5)

The workflow engine introduces a new CLI command that executes multi-step Databus pipelines defined as YAML files:

```
databusclient workflow run pipeline.yml
databusclient workflow run pipeline.yml --manifest run.json
```

A pipeline is a sequence of steps mapping directly to existing commands — `download`, `deploy`, or `delete` — executed sequentially with shared state, per-step error handling, and automatic manifest generation.

3.1 Workflow YAML Format:

```
manifest: run_manifest.json

steps:
- name: fetch_dataset
  command: download
  uri: https://databus.dbpedia.org/dbpedia/generic/labels/2023.12.01
  convert_to: bz2
  on_error: retry
  retry:
    max_attempts: 3
    delay_seconds: 5

- name: publish_dataset
  command: deploy
  version_id: https://databus.dbpedia.org/myaccount/research/labels/2024.01
  title: "Processed Labels"
  abstract: "Processed from DBpedia 2023.12.01"
  description: "Converted and redeployed labels dataset"
  license: https://creativecommons.org/licenses/by-sa/3.0/
  api_key: ${DATABUS_API_KEY}
  files: ${steps.fetch_dataset.output_files}
  on_error: fail
```

```

- name: cleanup_old
  command: delete
  uris:
    - https://databus.dbpedia.org/myaccount/research/labels/2023.old
  api_key: ${DATABUS_API_KEY}
  on_error: continue

```

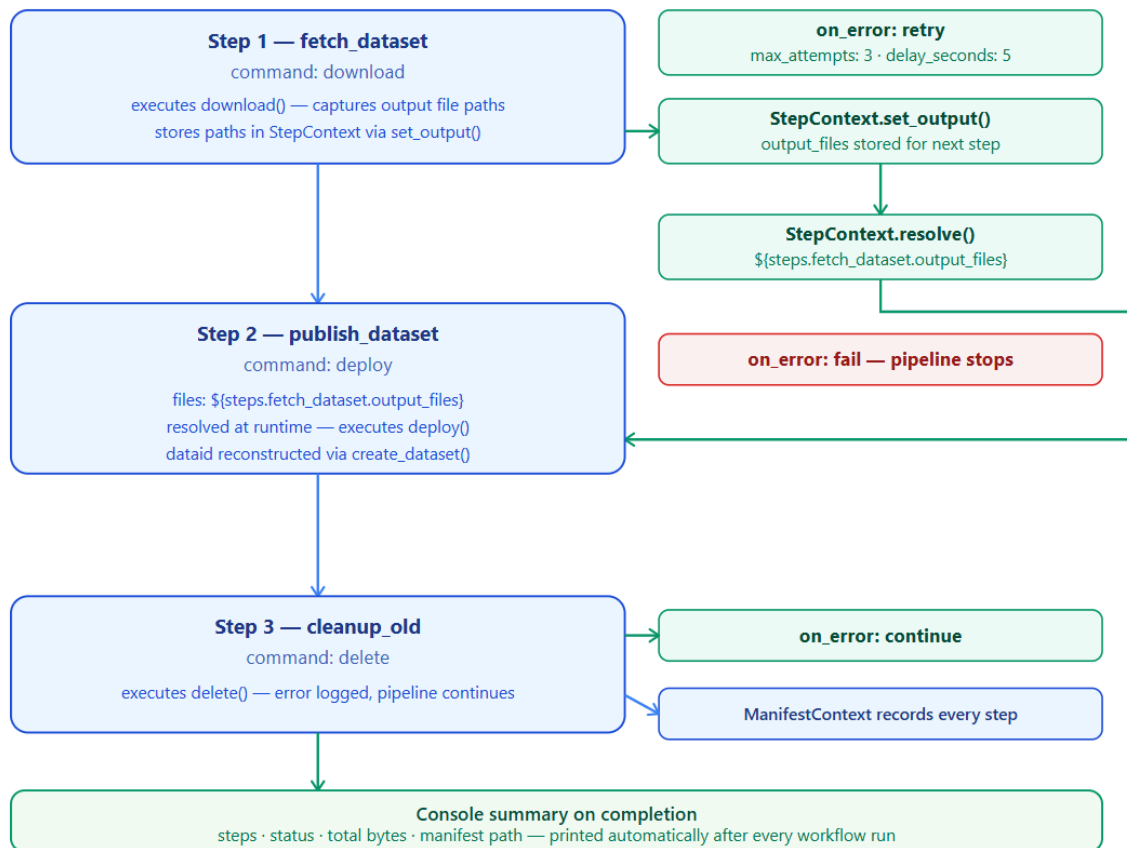
3.2 Step Chaining

The most technically significant feature of the workflow engine is step chaining — the ability for one step to consume the outputs of a previous step without manual intervention.

`files: ${steps.fetch_dataset.output_files}` is not a static value. It is a dynamic reference resolved at runtime by `StepContext.resolve()`. When `fetch_dataset` completes, `DownloadStep` captures the downloaded file paths and stores them in `StepContext` via `set_output()`. When `publish_dataset` begins, the engine resolves every parameter value through `StepContext` before execution — any `${steps.name.key}` reference is replaced with the actual stored value.

This means a full download-then-deploy pipeline can be defined without knowing in advance which files will be downloaded.

Fig. 3 — Workflow Engine: Step Chaining and Per-Step Error Recovery



3.3 Error Recovery

Each step declares its own failure behavior via `on_error`. Three modes are supported:

Mode	Behavior	Typical use
<code>fail</code>	Stop the pipeline immediately	Critical steps where failure invalidates the entire run
<code>continue</code>	Log the error and proceed to next step	Cleanup steps where failure is acceptable
<code>retry</code>	Retry up to <code>max_attempts</code> times with <code>delay_seconds</code> gap	Network operations subject to transient failures

The `retry` mode is particularly valuable for Databus operations, which involve network requests that can fail transiently due to timeouts or rate limits. Rather than failing an entire nightly pipeline on a single network hiccup, retry handles it transparently.

3.4 Environment Variable Substitution

Credentials are never hardcoded in workflow files. Any value written as `${VARIABLE_NAME}` is resolved from the environment at parse time. If the variable is not set, `WorkflowParser` raises a `MissingEnvVarError` before execution begins — the pipeline fails early with a clear message rather than mid-run with an authentication error.

3.5 Console Summary

After every workflow execution, a structured summary is printed automatically to console — no flag required:

```
Workflow completed - pipeline.yml
┌──────────┬──────────┬──────────┬──────────┐
│ fetch_dataset │ download │ ✓ success │          │
│ publish_dataset │ deploy  │ ✓ success │          │
│ cleanup_old   │ delete  │ ✗ failed (continued) │          │
└──────────┴──────────┴──────────┴──────────┘
Steps: 3 total · 2 succeeded · 1 failed
Manifest saved: run_manifest.json
```

The `STEP_REGISTRY` dictionary maps command names to step classes. Adding a new step type in future requires only adding a new class and one entry in the registry — the engine itself never needs to change.

Implementation Plan

The implementation touches four existing files and introduces two new modules. All changes to existing files are purely additive — no existing logic is removed or rewritten.

Existing Files Modified

File	Change
<code>databusclient/cli.py</code>	Add <code>--manifest</code> flag to <code>download</code> , <code>deploy</code> , and <code>delete</code> commands. Add <code>manifest</code> command group with <code>replay</code> and <code>summary</code> subcommands. Add <code>workflow</code> command group with <code>run</code> subcommand.
<code>databusclient/api/download.py</code>	Add optional <code>manifest_context=None</code> parameter. Call <code>manifest_context.record_file()</code> after each file operation.
<code>databusclient/api/deploy.py</code>	Add optional <code>manifest_context=None</code> parameter. Record deploy inputs and API response status.
<code>databusclient/api/delete.py</code>	Add optional <code>manifest_context=None</code> parameter. Record each URI and its deletion status.

All new logic lives in two new modules — `databusclient/manifest/` and `databusclient/workflow/` — plus `convert.py` for the format conversion layer.

How the Two Modules Connect

When a workflow run is called with `--manifest`, the engine creates a `ManifestContext` at the start of the run and passes it into each step alongside the `StepContext`. Every step records its file-level outcomes into `ManifestContext` as it executes. At the end of the run, `ManifestWriter` serializes the complete context to disk. The manifest therefore captures the entire workflow — not just individual commands — as one unified record.

When a workflow run is called without `--manifest`, `ManifestContext` is never created and nothing is written. The workflow executes identically.

Concrete Use Cases

- **Use Case 1: Reproducible Research Download** A researcher downloads a specific DBpedia version with `--manifest run.json`. Six months later, anyone can run `databusclient manifest replay run.json` on any machine to reproduce the exact same download and verify identical checksums. What previously required careful parameter documentation becomes a single replayable file.
- **Use Case 2: Automated Nightly Publishing Pipeline** A DBpedia extraction job runs nightly via cron. A workflow YAML defines `download` → `deploy` → `delete previous version`.

The pipeline runs unattended and produces a full manifest audit trail reviewable the next morning without touching server logs.

- **Use Case 3: Batch Deployment with Error Recovery** An organization deploys 20 dataset versions in one run. Retry logic handles transient network failures automatically. The manifest records exactly which versions succeeded and which failed. The console summary gives an immediate overview without log access.
- **Use Case 4: CI/CD Integration via GitHub Actions** A research team calls `databusclient workflow run publish.yml` as a post-extraction step in GitHub Actions. The manifest is saved as a build artifact. On failure, the team inspects it directly in the Actions UI to see exactly which step failed and why.
- **Use Case 5: Debugging a Failed Deployment** A deploy operation partially fails across multiple files. Because the manifest records structured error details — including the error message, traceback, and retry count — directly against each failed file entry, running `databusclient manifest summary run.json` gives an immediate, structured view of exactly what failed and why. Debugging drops from hours of log reading to a single command.

Project Timelines

Timeline	Tasks
Up until May 1 Codebase Study and Design Finalization	→ Study the existing codebase in depth — <code>cli.py</code>, <code>download.py</code>, <code>deploy.py</code>, <code>delete.py</code>. → Study <code>rdflib</code> — <code>Graph</code>, <code>ConjunctiveGraph</code>, supported formats, and isomorphic comparison. → Finalize manifest JSON-LD schema, workflow YAML spec, and conversion layer design with mentor. → Set up local development environment with real Databus datasets. Deliverables: All integration points understood. Manifest schema, workflow YAML spec, and conversion layer design finalized with mentor. Development environment ready.
May 1 – May 31 Environment Setup and Prototype Validation	→ Set up integration test infrastructure using real Databus datasets. → Study existing test suite in <code>tests/</code> to understand testing patterns and conventions. → Explore the existing CLI structure in <code>cli.py</code> to plan exact placement of new command groups.

	→ Prototype a minimal <code>ManifestContext</code> sketch to validate the threading approach before coding begins. Deliverables: Integration test environment ready. Prototype manifest context validated. Full clarity on implementation approach with no remaining doubts.
Week 1, 2 Manifest Module and Download Integration	→ Implement <code>ManifestContext</code> and <code>ManifestWriter</code> classes in <code>databusclient/manifest/</code>. → Integrate optional <code>manifest_context</code> parameter into <code>download()</code>. → Add <code>--manifest</code> flag to <code>download</code> CLI command Deliverables: Working <code>--manifest</code> flag on <code>download</code> command. <code>ManifestContext</code> and <code>ManifestWriter</code> fully tested.
Week 3, 4 Full Manifest Coverage and Summary Command	→ Extend <code>manifest_context</code> integration to <code>deploy()</code> and <code>delete()</code>. → Add <code>--manifest</code> flag to both CLI commands. → Implement <code>ManifestSchema</code> with versioning via <code>schema_version</code> field. → Implement <code>manifest summary</code> CLI subcommand Deliverables: <code>--manifest</code> working on all three commands. <code>manifest summary</code> command working. Schema versioning in place.
Week 5, 6 Manifest Replay and Mid-Term Milestone	→ Implement <code>ManifestReplayer</code> class in <code>manifest/replay.py</code>. → Handle all three command types in replay — <code>download</code> replay from saved params, <code>deploy</code> replay via <code>create_dataset()</code> reconstruction, <code>delete</code> replay defaulting to <code>dry_run=True</code> for safety. → End-to-end replay testing with real Databus datasets. Deliverables: <code>manifest replay</code> working correctly for all three command types. Safety guard for delete replay verified. Mid-term documentation ready.

Week 7, 8 Format and Mapping Conversion Layer	→ Implement <code>convert.py</code> — <code>convert_format()</code> for Layer 2 and <code>triples_to_quads()</code>, <code>quads_to_triples()</code>, <code>rdf_to_csv()</code>, <code>csv_to_rdf()</code> for Layer 3. → Add <code>--convert-format</code>, <code>--graph-name</code>, and <code>--base-uri</code> CLI flags to the download command. → Integrate <code>convert.py</code> into the download pipeline after decompression — composable with existing <code>--convert-to</code> and <code>--convert-from</code> flags. → Write <code>test_conversion.py</code> — 9 format round trip tests and 5 mapping round trip tests using <code>rdflib</code>'s isomorphic comparison. Deliverables: All four mapping directions implemented and tested. <code>--convert-format</code> flag working end-to-end with real Databus datasets. 14 round trip tests passing.
Week 9, 10 Workflow Engine Core and Step Execution	→ Implement <code>WorkflowParser</code> — YAML parsing, schema validation, environment variable substitution with <code>MissingEnvVarError</code> on missing variables. → Implement <code>WorkflowEngine</code> — execution loop and step sequencing. → Implement <code>DownloadStep</code>, <code>DeployStep</code>, <code>DeleteStep</code> and <code>STEP_REGISTRY</code>. → Basic end-to-end pipeline execution working without step chaining Deliverables: <code>databusclient workflow run pipeline.yml</code> executing all three step types end-to-end. Environment variable substitution working.
Week 11 Step Chaining and Dynamic Reference Resolution	→ Implement <code>StepContext</code> — output capture via <code>set_output()</code> and dynamic reference resolution via <code>resolve()</code>. → Integrate step chaining into <code>WorkflowEngine</code> — resolve all parameter values through <code>StepContext</code> before each step executes. → Test <code>\${steps.name.output_files}</code> resolution between download and deploy steps with real datasets.

	Deliverables: Step chaining working end-to-end. Download output files automatically passed as deploy input without user intervention.
Week 12 Error Recovery and Manifest Integration	→ Implement per-step error recovery — <code>fail</code>, <code>continue</code>, and <code>retry</code> modes. → Implement retry loop with configurable <code>max_attempts</code> and <code>delay_seconds</code>. → Integrate <code>ManifestContext</code> with <code>WorkflowEngine</code> — workflow run produces unified manifest automatically. → Implement console summary printed on workflow completion. → Full failure scenario testing including partial pipeline failures. Deliverables: All three error recovery modes working. Workflow run produces manifest automatically. Console summary working.
Week 13 Example Workflows and Integration Testing	→ Write and test all five example workflows against real Databus datasets. → Verify reproducible download, nightly pipeline, batch deployment, CI/CD, and debugging scenarios end-to-end. → Write <code>test_step_chaining.py</code> and <code>test_error_recovery.py</code> integration tests. Deliverables: Five tested example workflows in <code>examples/workflows/</code>. Full integration test suite passing.
Week 14 Documentation, Cleanup and Final Submission	→ Complete documentation — manifest schema reference, workflow YAML specification, API reference for new modules, updated README with usage examples. → Code cleanup and final mentor review. → Prepare final submission. Deliverables: Complete documentation. Clean release-ready codebase. Final backward-compatible release of updated Databus Python Client submitted.

Why me and my Motivation?

This project evolves the client from a purely command-driven utility into a reproducible, workflow-aware tool for DBpedia data operations. Researchers will be able to define, trace, and replay data pipelines in a single command. Automated publishing workflows will have a full audit trail. The client will become a first-class tool for production data release pipelines — not just a collection of individual commands.

My active contributions to the **Databus Python Client** — the v0.15 PyPI release, compression conversion pipeline, and Vault authentication improvements — gave me deep familiarity with the codebase internals and real-world usage gaps that this proposal directly addresses. The manifest and workflow ideas here are not abstract — they are solutions to problems I encountered while working with the client myself. I am committed to delivering this to a high standard and seeing it merged into the project.

References

[1] Frey et al. — Managing and Compiling Data Dependencies for Semantic Applications Using Databus Client — doi.org/10.1007/978-3-030-98876-0_10

[2] DBpedia Databus Java Client — github.com/dbpedia/databus-client

- **DBpedia Databus Python Client** — github.com/dbpedia/databus-python-client
- **JSON-LD 1.1 Specification — W3C** — w3.org/TR/json-ld11
- **DCAT Vocabulary Specification — W3C** — w3.org/TR/vocab-dcat-2
- **Click Documentation — Pallets Projects** — click.palletsprojects.com
- **pytest Documentation** — docs.pytest.org